



Tutorial: User-Defined Functions (UDFs)

User-Defined Functions (UDFs) allow users to define custom Column-based functions for transforming data in Spark.

There are performance concerns as UDFs are executed as a black box to Spark's [Catalyst Optimiser](#). The general rule of thumb is to use Spark SQL [Functions API](#) for querying data for better query optimisation.

However, here are 2 scenarios where UDFs might be useful:

1. Incorporate external libraries (e.g. SciPy's Fast Fourier Transform)
2. Split-Apply-Combine (GroupBy) operations e.g. find the set of months for each utilities account where consumption level is outside of its Inter-Quartile Range.



TIP

Pandas UDFs (`pyspark.sql.functions.pandas_udf`) are preferred over UDFs (`pyspark.sql.functions.udf`) as they are executed by Spark in batches using Apache Arrow & Pandas (with a default max of 10,000 records per batch). This allows for vectorized operations, whereas UDFs execute operations row-wise.

In our [Advanced Usage Notebook](#), we illustrate the difference in performance between the 3 methods and the differences in their Query Execution Plans.

Pandas UDF usage on GroupedData or over Window

Pandas UDFs can be used for aggregation on grouped data (i.e. summarise each group with a single value). Here we demonstrate getting the median travel time per originating station in the `lta_farecard` dataset.

`df` is a Spark DataFrame with `orig_loc` and `duration_s` columns in `string` and `integer` formats respectively.

Compute median farecard trip times with Pandas UDF

Here we retrieve the median travel time for each origin station by writing a Pandas UDF,

`get_median_travel_time`.

```
import pandas as pd
import pyspark.sql.functions as F

@F.pandas_udf("double")
def get_median_travel_time(duration: pd.Series) -> float:
    return duration.median()

# compute median travel time for each originating station
median_time_pudf = (
    df.groupBy("orig_loc")
      .agg(get_median_travel_time("duration_s"))
      .sort("orig_loc", ascending=True)
)

median_time_pudf.display()
```

❗ INFO

Spark 3.1.0 added `pyspark.sql.functions.percentile_approx`, which can be used to compute the median travel time per originating station within a specified relative error. However, a Pandas UDF should be used if the exact median is desired.

Pandas UDF with `applyInPandas`

Pandas UDF can also be used for custom operations on each set of grouped data. When used on `pyspark.sql.GrouppedData.applyInPandas()`, it can take multiple columns as input and return multiple columns with an arbitrary number of rows as output; the `pandas_udf` maps a `pd.DataFrame` to a `pd.DataFrame`. The schema needs to be specified. Here we demonstrate usage by finding overlapping trips from each vehicle in the `lta_taxi` dataset.

`df` is a Spark DataFrame with `VEHICLE_ID`, `ORIGIN_DATE_TIME_TS`, and `DEST_DATE_TIME_TS` columns in `string`, `integer` and `integer` formats respectively.

Find overlapping trips for each taxi vehicle

Here, the function `flag_overlap` adds an additional column, `OVERLAP` to `df` if a specific trip record overlaps with another trip by the same vehicle.

```
def flag_overlap(pdf: pd.DataFrame) -> pd.DataFrame:
    # sort each groupset
    pdf.sort_values(["ORIGIN_DATE_TIME_TS"], inplace=True)

    # get first row
    pd_iterator = pdf.itertuples()
    prev_max_row = next(pd_iterator)
    pdf.loc[prev_max_row.Index, "OVERLAP"] = False

    # from second row onwards
    for row in pd_iterator:
        # check for overlaps
        if row.ORIGIN_DATE_TIME_TS <= prev_max_row.DEST_DATE_TIME_TS:
            pdf.loc[row.Index, "OVERLAP"] = True
            pdf.loc[prev_max_row.Index, "OVERLAP"] = True
        else:
            pdf.loc[row.Index, "OVERLAP"] = False
        # if new max, replace old max
        prev_max_row = max(
            [prev_max_row, row],
            key=lambda r: r.DEST_DATE_TIME_TS
        )

    # only keep overlaps
    pdf = pdf[pdf.OVERLAP]
    return pdf
```

```
overlap_pudf = df.groupBy("VEHICLE_ID").applyInPandas(
    flag_overlap,
    schema=(
        "VEHICLE_ID string, ORIGIN_DATE_TIME_TS int, "
        "DEST_DATE_TIME_TS int, OVERLAP boolean"
    )
)
```